

COMSATS UNIVERSITY ISLAMABAD

ATTOCK CAMPUS



DEPARTMENT OF COMPUTER SCIENCE

SUBMITTED TO:	MAAM TAHIRA SADAF
COURSE	OPERATING SYSTEM

STUDENT DETAILS

SUBMITTED BY:	ATIKA EMAAN
REG NO:	FA24-BCS-065
ASSIGNMENT NO:	02

THREAD MODELS IN OPERATING SYSTEMS

1. INTRODUCTION TO THREADS

In old or basic operating systems, every process had only one thread. That means each program could only do one thing at a time. But in real life, programs need to do many things at the same time. For example, a word processor needs to take input from the keyboard, save the file, and format the text all at once. This is why threads were introduced.

Tanenbaum explains it like this:

"In traditional operating systems, each process has an address space and a single thread of control. Nevertheless, in many situations, it is desirable to have multiple threads of control in the same address space running in quasi-parallel, as though they were almost separate processes."

So basically, a thread is a smaller unit inside a process. Multiple threads can run inside one process and they all share the same memory. Think of a process as a factory and threads as the workers inside that factory. All workers share the same building but each worker does their own job.

Tanenbaum gives a simple definition:

"Processes are used to group resources together; threads are the entities scheduled for execution on the CPU."

1.1 Why Do We Need Threads?

Tanenbaum gives four main reasons why threads are useful:

Reason 1 Makes Programming Easier: When a program has to do many things at once, using threads makes it simple. Instead of writing complicated code to manage everything, we just create separate threads for each task. **"By decomposing such an application into multiple sequential threads that run in quasi-parallel, the programming model becomes simpler."**

Reason 2 Threads Are Faster to Create: Creating a thread is much faster than creating a full process. **"In many systems, creating a thread goes 10 to 100 times faster than creating a process."** This saves time when a program needs to quickly create and remove tasks.

Reason 3 Better Performance: When a program has both heavy calculations and input/output work, threads allow both to happen at the same time. **"When there is substantial computing and also substantial I/O, having threads allows these activities to overlap, thus speeding up the application."**

Reason 4 Works Well on Multi CPU Systems: On computers with more than one processor, different threads can run on different processors at the exact same time. **"Threads are useful on systems with multiple CPUs, where real parallelism is possible."**

1.2 What Does a Thread Look Like?

Each thread has its own private things and also shares some things with other threads in the same process.

Things shared by all threads (belonging to the process): Address space, Global variables, Open files, Child processes, Pending alarms, Signal handlers, Accounting information

Things private to each thread: Program counter, Registers, Stack, State

Tanenbaum explains this well:

"The thread has a program counter that keeps track of which instruction to execute next. It has registers, which hold its current working variables. It has a stack, which contains the execution history."

One important thing to understand is that because all threads share the same memory, one thread can accidentally damage another thread's data. Tanenbaum says:

"Since every thread can access every memory address within the process' address space, one thread can read, write, or even wipe out another thread's stack. There is no protection between threads."

This is not a bug. It is by design because all threads in a process belong to the same program and are supposed to work together, not against each other.

2. POSIX THREADS (PTHREADS)

Different operating systems were creating their own thread systems, which caused problems because code written for one system would not work on another. To fix this, a standard was created.

"To make it possible to write portable threaded programs, IEEE has defined a standard for threads in IEEE standard 1003.1c. The threads package it defines is called Pthreads. Most UNIX systems support it. The standard defines over 60 function calls."

The most important Pthreads functions are:

pthread_create Creates a new thread. **"This call is intentionally very much like the fork system call, with the thread identifier playing the role of the PID."**

pthread_exit Ends the calling thread. **"This call stops the thread and releases its stack."**

pthread_join Makes one thread wait for another thread to finish. **"The thread that is waiting calls pthread_join to wait for a specific other thread to terminate."**

pthread_yield A thread voluntarily gives up the CPU so another thread can run. This is important because threads do not have a timer to force them to stop, so they must be polite and give up the CPU on their own sometimes.

pthread_attr_init Creates and sets up a thread's settings like its priority and stack size.

pthread_attr_destroy Deletes the settings structure when it is no longer needed.

These functions form the basic toolkit for creating and managing threads in UNIX based systems.

3. THREAD IMPLEMENTATION MODELS

Now comes the most important part of this assignment. How are threads actually built inside an operating system? Tanenbaum describes three main ways to implement threads. Each method has its own advantages and disadvantages.

3.1 Model 1 User Level Threads

In this model, the operating system kernel does not know about threads at all. The threads are managed completely by a library running in user space.

"The first method is to put the threads package entirely in user space. The kernel knows nothing about them. As far as the kernel is concerned, it is managing ordinary, single-threaded processes. With this approach, threads are implemented by a library."

Think of it like this: the kernel thinks it is only dealing with one process, but inside that process, the program itself is managing multiple threads secretly.

Each process keeps its own thread table to track all the threads:

"When threads are managed in user space, each process needs its own private thread table to keep track of the threads in that process. This table is analogous to the kernel's process table, except that it keeps track only of the per-thread properties such as each thread's program counter, stack pointer, registers, and state."

Advantages of User-Level Threads:

- Very fast switching between threads. **"Doing thread switching like this is at least an order of magnitude faster than trapping to the kernel."** This is because switching happens without going into the kernel at all.
- No trap is needed, no context switch is needed, and the memory cache does not need to be cleared.

- Every process can have its own custom thread scheduling method.
- Works even on operating systems that do not support threads natively.
- Better scalability because kernel threads need kernel memory, but user-level threads do not.

Disadvantages of User-Level Threads:

The biggest problem is with blocking system calls. If one thread makes a call that causes it to wait, the entire process is frozen. Tanenbaum explains:

"Suppose that a thread reads from the keyboard before any keys have been hit. Letting the thread actually make the system call is unacceptable, since this will stop all the threads."

Another problem is page faults. If one thread needs a memory page that is not loaded yet:

"If a thread causes a page fault, the kernel, unaware of even the existence of threads, naturally blocks the entire process until the disk I/O is complete, even though other threads might be runnable."

There is also no preemption. If one thread never stops running:

"If a thread starts running, no other thread in that process will ever run unless the first thread voluntarily gives up the CPU. Within a single process, there are no clock interrupts, making it impossible to schedule processes round-robin fashion."

Finally, user-level threads are actually a poor match for the most common use case:

"Programmers generally want threads precisely in applications where the threads block often, as for example in a multithreaded Web server. Once a trap has occurred to the kernel to carry out the system call, it is hardly any more work for the kernel to switch threads if the old one has blocked."

3.2 Model 2 Kernel-Level Threads

In this model, the operating system kernel knows about and manages all the threads. There is no thread library running secretly in user space.

"Now let us consider having the kernel know about and manage the threads. No run-time system is needed in each process. Instead, the kernel has a thread table that keeps track of all the threads in the system. When a thread wants to create a new thread or destroy an existing thread, it makes a kernel call, which then does the creation or destruction by updating the kernel thread table."

The kernel thread table stores information about every thread:

"The kernel's thread table holds each thread's registers, state, and other information. The information is the same as with user-level threads, but now kept in the kernel instead of in user space."

Advantages of Kernel-Level Threads:

- When a thread blocks, the kernel can simply run another thread. "When a thread blocks, the kernel, at its option, can run either another thread from the same process or a thread from a different process."
- Kernel threads do not require any new nonblocking system calls.
- If one thread causes a page fault, other threads in the same process can still run.
- Old threads can be recycled instead of deleted. "When a thread is destroyed, it is marked as not runnable, but its kernel data structures are not otherwise affected. Later, when a new thread must be created, an old thread is reactivated, saving some overhead."

Disadvantages of Kernel-Level Threads:

- Creating and destroying threads is much slower because it requires a system call every time.
- "The cost of a system call is substantial, so if thread operations are common, much more overhead will be incurred."
- Problems with fork: "What happens when a multithreaded process forks? Does the new process have as many threads as the old one did, or does it have just one?"
- Signal handling becomes complicated because signals are sent to processes, not to individual threads.

3.3 Model 3 Hybrid Threads

This model tries to get the best of both worlds by combining user-level and kernel-level threads together.

"Various ways have been investigated to try to combine the advantages of user-level threads with kernel-level threads. One way is to use kernel-level threads and then multiplex user-level threads onto some or all of them."

"When this approach is used, the programmer can determine how many kernel threads to use and how many user-level threads to multiplex on each one. This model gives the ultimate in flexibility."

How it works in practice:

"The kernel is aware of only the kernel-level threads and schedules those. Some of those threads may have multiple user-level threads multiplexed on top of them. These user-level threads are created, destroyed, and scheduled just like user-level threads in a process that runs on an operating system without multithreading capability."

So for example, you might have 4 kernel threads but 20 user-level threads. The 20 user threads take turns running on top of the 4 kernel threads. This way you get fast user-level switching most of the time, but you also avoid completely blocking the whole process when one thread makes a blocking call.

3.4 Scheduler Activations

This is a clever design idea that tries to make user-level threads work as well as kernel-level threads. The key idea is communication between the kernel and the user-level thread manager.

"The goals of the scheduler activation work are to mimic the functionality of kernel threads, but with the better performance and greater flexibility usually associated with threads packages implemented in user space."

The mechanism used is called an upcall:

"When the kernel knows that a thread has blocked, the kernel notifies the process' run-time system, passing as parameters on the stack the number of the thread in question and a description of the event that occurred. The notification happens by having the kernel activate the run-time system at a known starting address, roughly analogous to a signal in UNIX. This mechanism is called an upcall."

When the run-time system receives this upcall it wakes up another thread:

"The run-time system can reschedule its threads, typically by marking the current thread as blocked and taking another thread from the ready list, setting up its registers, and restarting it."

This solves the blocking problem of user-level threads while keeping the speed advantage.

3.5 Pop-Up Threads

This is a special type of thread used mostly in networked and distributed systems. Instead of having a thread already waiting for incoming messages, the system creates a brand new thread every time a message arrives.

"A completely different approach is also possible, in which the arrival of a message causes the system to create a new thread to handle the message. Such a thread is called a pop-up thread. A key advantage of pop-up threads is that since they are brand new, they do not have any history

registers, stack, whatever — that must be restored. Each one starts out fresh and each one is identical to all the others. This makes it possible to create such a thread quickly."

The benefit is very fast response time: **"The result of using pop-up threads is that the latency between message arrival and the start of processing can be made very short."**

4. THREAD IMPLEMENTATION IN LINUX

Linux has a very unique way of handling threads that is different from most other operating systems.

4.1 Linux Treats Threads and Processes the Same Way

In most operating systems, threads and processes are clearly different things. Linux blurs this line completely.

"The Linux kernel treats both processes and threads simply as executable entities, and will schedule them based on a global scheduling policy."

Linux does not have a separate concept for threads. Instead, it uses one powerful system call that can create either a thread or a process depending on how you use it.

"In 2000, Linux introduced a powerful new system call, clone, that blurred the distinction between processes and threads and possibly even inverted the primacy of the two concepts. Clone is not present in any other version of UNIX."

4.2 The clone() System Call

The clone() call is the heart of Linux thread creation. By setting different flags, you can decide exactly what the new thread shares with the old one.

"The call creates a new thread, either in the current process or in a new process, depending on sharing_flags. If the new thread is in the current process, it shares the address space with the existing threads, and every subsequent write to any byte in the address space by any thread is immediately visible to all the other threads in the process."

The sharing flags from Figure 10-9 of the book:

CLONE_VM If set, the new thread shares the address space (same process). If cleared, it gets its own address space (new process).

CLONE_FS If set, share the working directory and root directory. If cleared, do not share them.

CLONE_FILES set, share file descriptors. If cleared, copy them.

CLONE_SIGHAND If set, share the signal handler table. If cleared, copy it.

CLONE_PARENT If set, the new thread has the same parent as the caller. If cleared, the caller becomes the parent.

Tanenbaum explains why this level of control is possible:

"This fine-grained sharing is possible because Linux maintains separate data structures for the various items. The task structure just points to these data structures, so it is easy to make a new task structure for each cloned thread and have it point either to the old thread's data structures or to copies of them."

4.3 PID and TID in Linux

To stay compatible with standard UNIX while also supporting this flexible thread model, Linux uses two different identifiers:

"Linux distinguishes between a process identifier (PID) and a task identifier (TID). Both fields are stored in the task structure. When clone is used to create a new process that shares nothing with its creator, PID is set to a new value; otherwise, the task receives a new TID, but inherits the PID. In this manner all threads in a process will receive the same PID as the first thread in the process."

This means all threads in the same process look like one process to the outside world, which keeps UNIX compatibility intact.

4.4 Problems with Fork and Threads in Linux

When threads and the fork() call are combined, tricky problems arise:

"The main issue in introducing threads is maintaining the correct traditional UNIX semantics. Suppose that a process with multiple kernel threads does a fork system call. Should all the other threads be created in the new process?"

If some threads are not copied to the child process, there can be a deadlock:

"Suppose that one of the not-created threads holds a mutex that the one-and-only thread in the new process tries to acquire after doing the fork. The mutex will never be released and the one thread will hang forever."

4.5 How Linux Schedules Threads

Since Linux treats threads as the basic unit of execution, scheduling also happens at the thread level:

"Linux threads are kernel threads, so scheduling is based on threads, not processes. Linux distinguishes three classes of threads for scheduling purposes: Real-time FIFO, Real-time round robin, and Timesharing."

For priority levels:

"Linux internally distinguishes among 140 priority levels. Tasks running at priority level 100 will receive time quanta of 800 msec, whereas tasks at priority level 139 will receive 5 msec."

Higher priority threads get more CPU time, and lower priority threads get less. This gives the system flexibility to handle both real-time and normal tasks together.

5. THREAD IMPLEMENTATION IN WINDOWS

Windows handles threads in a very different way compared to Linux and UNIX. Tanenbaum explains this in Section 11.4 of the book.

5.1 Basic Idea of Windows Threads

In Windows, a process is just a container. It holds resources. The actual work is done by threads.

"In Windows processes are containers for programs. They hold the virtual address space, the handles that refer to kernel-mode objects, and threads."

"Threads are the kernel's abstraction for scheduling the CPU in Windows. Priorities are assigned to each thread based on the priority value in the containing process. Threads can also be affinitized to run only on certain processors."

Each thread has two stacks:

"Each thread has two separate call stacks, one for execution in user mode and one for kernel mode. There is also a TEB (Thread Environment Block) that keeps user-mode data specific to the thread, including per-thread storage."

5.2 Why Windows Was Designed Differently from UNIX

The reason Windows and UNIX have different thread designs comes from their history. UNIX was built for small, simple machines. Windows NT was built for powerful multi-processor machines from the very beginning.

"UNIX was designed for 16-bit single-processor systems that used swapping to share memory among processes. Having the process as the unit of concurrency and using fork was a brilliant idea for those systems."

"In contrast, the hardware environment at the time Cutler's team wrote NT was 32-bit multiprocessor systems. NT used processes as containers for sharing memory and object resources, and used threads as the unit of concurrency for scheduling."

This is why Windows uses threads as the core scheduling unit while UNIX traditionally focused more on processes.

5.3 The Hierarchy: Jobs, Processes, Threads, Fibers

Windows has four levels of managing concurrent work:

"Jobs contain processes. Processes contain threads. But threads do not contain fibers. The relationship of threads to fibers is normally many-to-many."

From Figure 11-23 of the book:

Job --- A group of processes that share limits and quotas. Used in AppContainers.

Process ---A container that holds resources like memory and file handles.

Thread--- The actual unit that the kernel schedules on the CPU.

Fiber ---A very lightweight thread that is managed completely in user space. Rarely used.

Thread Pool --- A task-based programming model built on top of threads.

User-Mode Thread --- An extension that allows switching threads without entering the kernel.

5.4 Fibers Very Lightweight Threads

Fibers are even lighter than threads because switching between them does not need kernel involvement:

"The primary advantage of fibers is that the overhead of switching between fibers is much lower than switching between threads. A thread switch requires entering and exiting the kernel. A fiber switch saves and restores a few registers without changing modes at all."

However fibers have serious problems:

"Most of the Win32 libraries are completely unaware of fibers, and applications that attempt to use fibers as if they were threads will encounter various failures. The kernel has no knowledge of fibers."

Because of these problems fibers are rarely used in modern Windows applications.

5.5 Thread Pools

Creating a new thread every time you need to do a small task is wasteful. Thread pools solve this:

"Thread creation is too expensive to be invoked every time a program wants to execute a small task concurrently. An alternative approach is for a program to allocate a limited

number of threads, and maintain a queue of tasks that need to be run. As a thread finishes the execution of a task, it takes another one from the queue. Windows formalizes this solution into the Win32 thread pool."

5.6 User-Mode Scheduling :

Windows 7 introduced UMS to allow faster thread switching without always going into the kernel:

"UMS is similar to facilities used in other operating systems, such as scheduler activations. It can be used to switch between user threads without first having to enter the kernel, providing the benefits of fibers, but with much better integration into Win32."

The three parts of UMS:

Part 1 User-mode switching: "A user-mode scheduler can be written to switch between user threads without entering the kernel. When a user thread does enter kernel mode, UMS will find the corresponding kernel thread and immediately switch to it."

Part 2 Re-entering the scheduler: "When the execution of a kernel thread blocks to await the availability of a resource, UMS switches to a special user thread and executes the user-mode scheduler so that a different user thread can be scheduled to run on the current processor."

Part 3 System-call completion: "After a blocked kernel thread is finished, a notification containing the results of the system calls is queued for the user-mode scheduler so that it can switch to the corresponding user thread next time it makes a scheduling decision."

6. PROBLEMS WHEN CONVERTING SINGLE-THREADED CODE TO MULTI-THREADED

Many old programs were written with only one thread in mind. Making them work with multiple threads is harder than it looks.

6.1 Global Variable Problem

The most common problem is that multiple threads share the same global variables and can overwrite each other's data.

Tanenbaum uses the `errno` variable as an example:

"Thread 1 executes the system call access to find out if it has permission to access a certain file. The operating system returns the answer in the global variable `errno`. After control has returned to thread 1, but before it has a chance to read `errno`, the scheduler switches to

thread 2. Thread 2 executes an open call that fails, which causes errno to be overwritten and thread 1's access code to be lost forever."

The solution is: "Assign each thread its own private global variables. In this way, each thread has its own private copy of errno and other global variables, so conflicts are avoided."

6.2 Non-Reentrant Library Functions

Some library functions were written assuming only one thread would use them at a time. When multiple threads call them, chaos can happen:

"Many library procedures are not reentrant. Memory-allocation procedures such as malloc in UNIX maintain crucial tables about memory usage. While malloc is busy updating these lists, they may temporarily be in an inconsistent state, with pointers that point nowhere. If a thread switch occurs while the tables are inconsistent and a new call comes in from a different thread, an invalid pointer may be used, leading to a program crash."

6.3 Signal Handling Problems

Signals were designed for single-threaded processes, so they cause confusion in multi-threaded programs:

"Some signals are logically thread specific, whereas others are not. When threads are implemented entirely in user space, the kernel does not even know about threads and can hardly direct the signal to the right one."

6.4 Stack Management Problems

Each thread needs its own stack, but the kernel may not know about all these stacks:

"When a process has multiple threads, it must also have multiple stacks. If the kernel is not aware of all these stacks, it cannot grow them automatically upon stack fault."

7. ANALYTICAL JUSTIFICATION

7.1 No Single Thread Model is Perfect for Every Situation

After studying all the models, it is clear that each one has a specific situation where it works best. There is no single best solution.

The user-level model is fastest to switch between threads but fails badly when threads block on I/O. The kernel-level model handles blocking correctly and supports true parallelism on multiple processors, but is slower to create and switch threads. The hybrid model tries to combine both but adds complexity.

Tanenbaum himself summarizes this tradeoff clearly through his three-way comparison of server designs: threads give parallelism with simple blocking calls, single-threaded servers are simple but slow, and finite-state machines give performance without threads but are very complex to write and maintain.

7.2 Linux Made the Right Choice for Flexibility

Linux's choice to use `clone()` with flags instead of a separate thread concept gives programmers maximum control. A programmer can choose exactly what to share and what not to share between execution units. This makes Linux both POSIX-compatible and highly flexible. The copy-on-write mechanism also means that creating new threads or processes is very cheap in practice because memory pages are only copied when actually written.

7.3 Windows Made the Right Choice for Multi-Processor Systems

Windows was designed from the start for multi-processor hardware. By making threads the fundamental scheduling unit and processes just resource containers, Windows is naturally good at parallel computing. The later addition of UMS shows that even Windows recognized the performance cost of always going through the kernel, and added a way around it for advanced applications.

7.4 The Web Server Example Shows Why Threads Matter

Tanenbaum's web server example is the clearest practical demonstration of why thread models matter. A single-threaded web server sits idle while waiting for the disk. A multithreaded server lets one thread wait for disk while another handles a new request. The performance difference is enormous in real applications. This practical example shows that thread model choice is not just a theoretical concern but a real engineering decision with major performance consequences.

8. CONCLUSION

Threads are one of the most important concepts in operating systems. They allow programs to do multiple things at the same time without creating heavy and slow separate processes.

As Tanenbaum shows throughout *Modern Operating Systems*, there is no single perfect thread model. User-level threads are fast but cannot handle blocking well. Kernel-level threads handle everything correctly but are slower. Hybrid models and scheduler activations try to get the best of both worlds. Pop-up threads are useful in network systems where fast response to messages is needed.

Linux takes a unique approach by treating processes and threads the same way and using the `clone()` system call with flags to control sharing. Windows takes a different but equally valid approach by making threads the core scheduling unit and processes just containers, with extras like fibers, thread pools, and UMS for advanced use cases.

The most important lesson from Tanenbaum is that the right thread model depends on what the application needs. If it does heavy computation with little I/O, user-level threads may be enough. If it handles many network connections with lots of blocking, kernel-level threads are better. Understanding these trade-offs is what separates a good system designer from an average one.

Reference:

- Modern Operating Systems Author: Andrew S. Tanenbaum
-